



FULL-STACK APPLICATION DESIGN AND DEVELOPMENT

COURSE II - NOSQL



INTRODUCTION TO NOSQL

- NoSQL (Not Only SQL) databases are designed to handle large volumes of unstructured and semi-structured data. Unlike traditional relational databases that rely on fixed schemas and tables, NoSQL offers flexible data models and supports horizontal scaling.
- This makes them well-suited for modern applications that require high performance, scalability, and the ability to manage diverse data types efficiently.

INTRODUCTION TO NOSQL

- **Key Features of NoSQL Databases**
- **Dynamic schema:** Allow flexible shaping of data to meet new requirements without the need to migrate or change schemas.
- **Horizontal scalability:** They scale horizontally for adding more nodes into the existing ones and acquire enough storage for even bigger datasets and much higher traffic by distributing the load on multiple servers.
- **Document-based:** Data is presented in flexible, semi-structured formats like JSON/BSON (e.g., MongoDB).
- **Key-value-based:** They possess a simple but fast access pattern (e.g., Redis) by storing data as pairs of keys and values.
- **Column-based:** Data is organized into columns instead of rows (e.g., CASSANDRA).
- **Distributed and high availability:** They are designed to be highly available and to automatically handle node failures and data replication across multiple nodes in a database cluster.
- **Flexibility:** Allow developers to store and retrieve data in a flexible and dynamic manner, with support for multiple data types and changing data structures.
- **Performance:** Perfect for big data and real-time analytics and high volume applications.

INTRODUCTION TO NOSQL

- **Challenges of NoSQL Databases**
- **Lack of standardization:** NoSQL systems can be vastly different from one another, making it even harder to choose the right one for a specific use case.
- **Lack of ACID compliance:** NoSQL databases may not provide consistency, which is a disadvantage for applications that need strict data integrity.
- **Narrow focus:** Great for storage but lack functionalities as transaction management, in which relational databases are great.
- **Absence of Complex Query Support:** They are not designed to handle complex queries, which means that they are not a good fit for applications that require complex data analysis or reporting.
- **Management complexity:** For large datasets, maintaining a NoSQL database could be quite more complicated than managing a relational database.
- **Limited GUI Tools:** While some NoSQL databases, like MongoDB offer GUI tools like MongoDB Compass, not all NoSQL databases provide flexible or user-friendly GUI tools.

SQL VS. NOSQL

Feature	SQL (Relational DB)	NoSQL (Non-Relational DB)
Data Model	Structured, Tabular	Flexible (Documents, Key-Value, Graphs)
Scalability	Vertical Scaling	Horizontal Scaling
Schema	Predefined	Dynamic & Schema-less
ACID Support	Strong	Limited or Eventual Consistency
Best For	Transactional applications	Big data, real-time analytics
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Cassandra, Redis

SQL VS. NOSQL

- Here is a simple example of how a structured data with rows and columns vs. a non-structured data without definition might look like. A product table in SQL DB might accept data looking like this:

```
{
  "id": "101",
  "category": "food",
  "name": "Apples",
  "qty": 150
}
```

- Whereas an unstructured NOSQL DB might save the products in many variations without constraints to change the underlying table structure:

```
Products=[
{
  "id": "101",
  "category": "food",
  "name": "California Apples",
  "qty": 150
},
{
  "id": "102",
  "category": "electronics",
  "name": "Apple MacBook Air",
  "qty": 10,
  "specifications": {
    "storage": "256GB SSD",
    "cpu": "8 Core",
    "camera": "1080p FaceTime HD camera"
  }
}]
```

SQL VS. NOSQL

- **When to Choose SQL?**
- SQL databases are well-suited for use cases where:
 - **Data consistency** and **transactional integrity** are critical (e.g., banking systems, customer relationship management).
 - The application needs a **well-defined schema** and structured data.
 - Complex queries and **relational data** are involved.
 - Applications requiring **multi-row transactions** (such as inventory management).
- **When to Choose NoSQL?**
- NoSQL databases are a better choice when:
 - You need to handle **large, unstructured data** sets, like social media data or logs.
 - The application requires **horizontal scalability** to accommodate high traffic and big data.
 - There is a need for **real-time data processing** and **flexible data models** (e.g., a content management system).
 - You are dealing with applications requiring **frequent changes in data structures**.

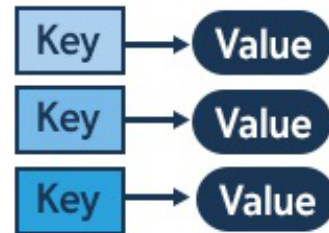
TYPES OF NOSQL DATABASES

- Databases store organized data for easy access and management. NoSQL databases handle large volumes of unstructured and semi-structured data, offering the scalability and flexibility today's diverse workloads demand.
- NoSQL databases can be classified into **four main types**, based on their data storage and retrieval methods:
 - Document-based databases
 - Key-value stores
 - Column-oriented databases
 - Graph-based databases
- Each type has unique advantages and use cases, making NoSQL a preferred choice for big data applications, real-time analytics, cloud computing and distributed systems.

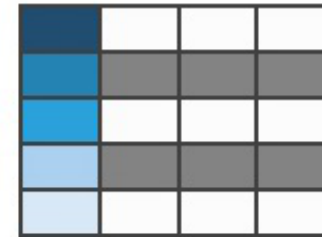
TYPES OF NOSQL DATABASES

NoSQL

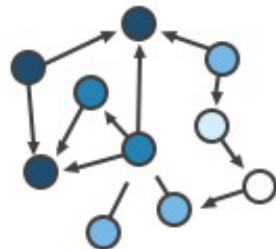
Key-Value



Column-Family



Graph



Document



TYPES OF NOSQL DATABASES

I. Document-Based Database

- A document-based database is a type of nonrelational database. Instead of organizing data into rows and columns like a relational database, it stores data as documents. These documents are typically formatted in JSON, BSON, or XML.
- Document databases allow data to be stored and retrieved in a structure that closely matches the data objects used in applications, which reduces the need for complex data translation. Individual elements within a document can be accessed efficiently using indexed fields, enabling faster queries.
- A collection is a group of documents with similar content. However, documents within a collection do not need to follow a strict schema, as document databases support flexible and evolving schema.

TYPES OF NOSQL DATABASES

- **Key features of document databases:**

- **Flexible schema:** Documents in a document database do not need to follow the same structure. Each document can have a different schema, allowing easy changes and evolution of data over time.
- **Faster creation and maintenance:** Documents are easy to create, and minimal maintenance is required after creation compared to traditional relational databases.
- **No foreign keys:** Document databases do not rely on foreign key relationships. Documents are generally independent, and relationships are handled through embedded data or application logic.
- **Open formats:** Documents are stored using open and widely supported formats such as JSON, XML, or BSON.

TYPES OF NOSQL DATABASES

- **Popular Document Databases & Use Cases**

Database	Use Case
MongoDB	Content management, product catalogs, user profiles
CouchDB	Offline applications, mobile synchronization
Firebase Firestore	Low-latency apps, chat applications

TYPES OF NOSQL DATABASES

2. Key-Value Stores

- The simplest form of a NoSQL database is a **key-value store**. Every data element in the database is stored in key-value pairs. Data can be retrieved by using a unique key allotted to each element in the database. The values can be simple data types like strings, numbers or complex objects. A key-value store is like a relational database with only two columns: the key and the value.
- **Key features of the key-value store:**
 - **Simplicity:** Data retrieval is extremely fast due to direct key access.
 - **Scalability:** Designed for horizontal scaling and distributed storage.
 - **Speed:** Ideal for caching and real-time applications.

TYPES OF NOSQL DATABASES

- **Popular Key-Value Databases & Use Cases**

Database	Use Case
Redis	Caching, real-time leaderboards, session storage
Memcached	High-speed in-memory caching
Amazon DynamoDB	Cloud-based scalable applications

TYPES OF NOSQL DATABASES

3. Column Oriented Databases

- A column-oriented database is a non-relational database that stores data in columns instead of rows. So when we want to run analytics on a small number of columns, we can read those columns directly without consuming memory with unwanted data. Column-oriented databases are designed to read data more efficiently and retrieve the data with greater speed.
- **Key features of column-oriented databases:**
 - **High scalability:** Supports distributed data processing.
 - **Compression:** Column-oriented storage enables efficient data compression.
 - **Faster query performance:** Best for analytical queries.

TYPES OF NOSQL DATABASES

- **Popular Column-Oriented Databases & Use Cases**

Database	Use Case
Apache Cassandra	Real-time analytics, IoT applications
Google Bigtable	Large-scale machine learning, time-series data
HBase	Distributed storage

TYPES OF NOSQL DATABASES

4. Graph-Based Databases

- Graph-based databases focus on the relationship between the elements. It stores data in the form of nodes in the database. The connections between the nodes are called links or relationships, making them ideal for complex relationship-based queries.
- Data is represented as nodes (objects) and edges (connections).
- Fast graph traversal algorithms help retrieve relationships quickly.
- Used in scenarios where relationships are as important as the data itself.
- **Key features of graph databases:**
 - **Relationship-centric storage:** Perfect for social networks, fraud detection, recommendation engines.
 - **Real-time query processing:** Queries return results almost instantly.
 - **Schema flexibility:** Easily adapts to evolving relationship structures.

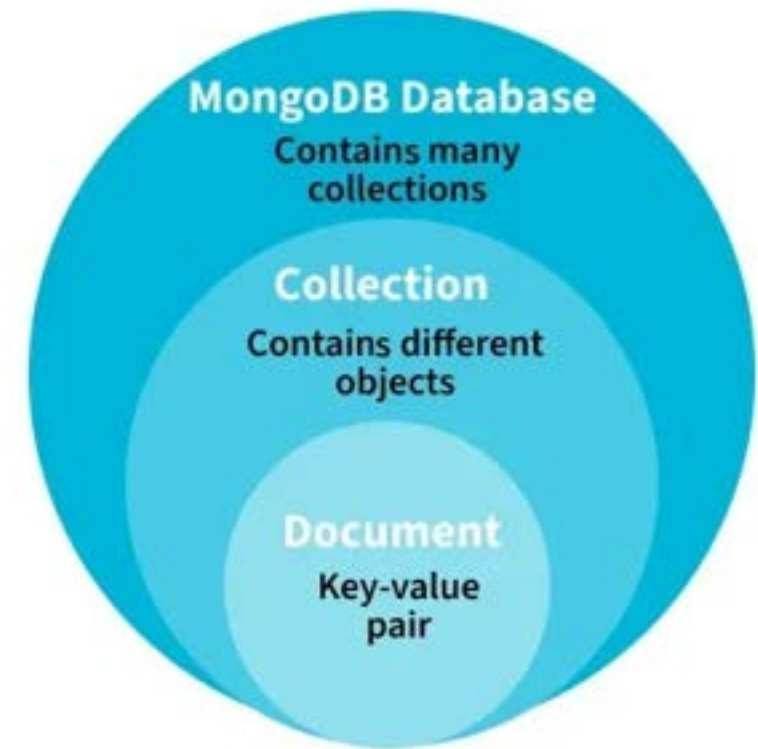
TYPES OF NOSQL DATABASES

- Popular Graph Databases & Use Cases

Database	Use Case
Neo4j	Fraud detection, social networks
Amazon Neptune	Knowledge graphs, AI recommendations
ArangoDB	Multi-model database, cybersecurity

NOSQL - MONGODB

- MongoDB is a NoSQL database that stores data in a flexible structure. A database is like a container for data, a collection is a group of related records, and a document is a single record stored in JSON-like format.
 - MongoDB can have many databases on one server, each separate from the others.
 - It creates three default databases: admin, local, and config (used internally).
 - A new database is made automatically when you insert data into it or when you write use <dbname>.



NOSQL - MONGODB

- **Databases in MongoDB**
- To see how many databases are present in your MongoDB server, write the following statement in the mongo shell:
`show dbs`
- The three default databases should be: admin, config and local.
- In order to create a new database:
`use myDB`
- The new database won't appear after a `SHOW` instruction, unless we add data into it.
- Naming restrictions for databases:
 - Database names must be case-insensitive.
 - The names cannot contain special characters such as `/`, `.`, `$`, `*`, `|`, etc.
 - MongoDB database names cannot contain null characters.
 - Database names cannot be empty and must contain less than 64 characters.

NOSQL - MONGODB

- **Collections in MongoDB**
- A Collection in MongoDB holds a group of documents and is a part of a database. Collections provide structure to data, but like the rest of MongoDB, they are schema-less.
 - **Schema-less:** MongoDB collections can store documents with different structures; documents in the same collection don't need to follow a uniform schema.
 - **Multiple collections:** A database can have multiple collections, each holding different types of documents.
- Naming restrictions for collections:
 - Collection names must start with an underscore (`\_`) or a letter (a-z or A-Z).
 - Collection names must not start with a number, must not contain \$, empty strings, null characters and must not begin with the prefix `system`, as this is reserved for MongoDB system collections.
 - The maximum length of a collection name must not exceed 120 bytes (including the database name, dot separator, and the collection name).

NOSQL - MONGODB

- **Creating a collection**

- After creating the database, we can add a collection to store documents. The collection is created using the following syntax:

```
db.collection_name.insertOne({..})
```

- Here, the insertOne() function is used to store data in the specified collection.

- **Example:**

```
db.Books.insertOne({ title: "Learn MongoDB", author: "Jane Doe", year: 2025 })
```

NOSQL - MONGODB

- **Documents in MongoDB**

- In MongoDB, the data records are stored as BSON documents. Here, BSON stands for binary representation of JSON documents, although BSON contains more data types compared to JSON. The document is created using field-value pairs or key-value pairs and the value of the field can be of any BSON type.

Syntax:

```
{  
  field1: value1  
  field2: value2  
  
  ...  
  fieldN: valueN  
}
```

- **Document structure:**

- A document in MongoDB is a flexible data structure made up of field-value pairs. For instance:

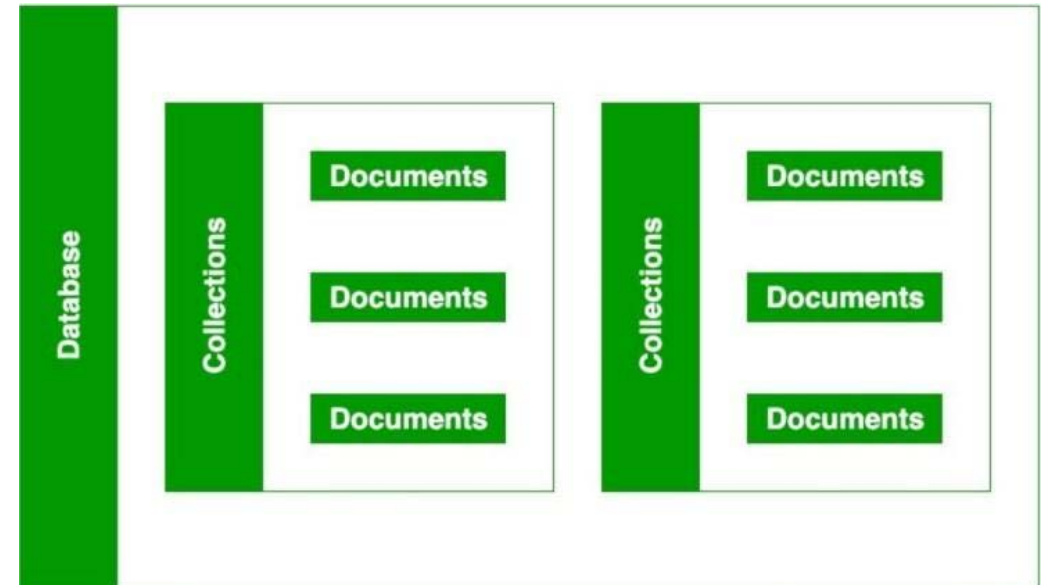
```
{  
  title: "MongoDB Basics",  
  author: "John Doe",  
  year: 2025  
}
```

NOSQL - MONGODB

- Naming restriction for document fields:
 - Fields in documents must be named with strings.
 - The `_id` field name is reserved to use as a primary key. And the value of this field must be unique, immutable, and can be of any type other than an array.
 - The field name cannot contain null characters.
 - The top-level field names should not start with a dollar sign (\$).
- **`_id` field in MongoDB**
- Every document must have a unique `_id` field, acting as a primary key.
 - **Automatic ObjectId:** If `_id` is not provided, MongoDB generates a unique ObjectId.
 - **Custom `_id`:** Users can assign a custom unique value for `_id`.

NOSQL - MONGODB

- MongoDB organizes data hierarchically into databases, collections, and documents, each serving a distinct purpose.
 - **Database:** A container for collections, providing structure and logical isolation for data.
 - **Collection:** A group of documents within a database, similar to a table in relational databases.
 - **Document:** A single data record within a collection, stored as a BSON object.



NOSQL - MONGODB

- Here's how you can create a database, collection, and document in MongoDB:

1. **Create or Switch to a Database**

```
use LibraryDB
```

2. **Create a Collection and Insert a Document**

```
db.Books.insertOne({  
  title: "MongoDB for Beginners",  
  author: "Alice Johnson",  
  year: 2025  
})
```

3. **Verify the Insertion**

```
db.Books.find()
```

- This will display the document stored in the Books collection within the LibraryDB database.

NOSQL - MONGODB

- In MongoDB, data types define the kind of data a field can hold, such as text, numbers, dates, or binary data. Below are the most commonly used MongoDB data types:

1. **String** – text data (UTF-8).
2. **Integer** – whole numbers (32-bit or 64-bit).
3. **Double** – decimal numbers.
4. **Boolean** – true/false.

- **Example:**

```
db.Student.insertOne({ name: "John Doe", age: 30, mark: 89.5, passed: true })
```

NOSQL - MONGODB

5. **Null** – no value.

6. **Array** – list of values.

■ **Example:**

```
db.Student.insertOne({ name: "John", phone_number: null, skills: ["JS", "Python"] })
```

7. **Object (embedded doc)** – nested document.

■ **Example:**

```
db.Book.insertOne({ book: { title: "The C Programming Language", author: "XYZ" } })
```

9. **Undefined** – undefined value (rare).

10. **Binary Data** – non-text data (e.g., files).

■ **Example:**

```
db.Project.insertOne({ name: "FauG", duration: undefined, binaryValue:"110011001" })
```

NOSQL - MONGODB

- A **MongoDB query** is a request to the database to retrieve **specific documents** or data based on certain conditions or criteria. It is similar to **SQL queries** in traditional **relational databases**, but MongoDB queries are written using **JavaScript-like syntax**. The most common query operation in **MongoDB** is the **find() method**, which allows users to **fetch documents** from a collection that match the given query criteria.
- To run a **MongoDB query**, you use the `db.collection_name.find()` function. You can specify **criteria** (conditions) within the `find()` method to filter documents based on field values. MongoDB supports a wide range of **query operators** to create complex queries, such as `$eq`, `$gte`, `$lte`, `$in`, `$or`, `$and`, and more.
- **Syntax:**
`db.collection_name.find({ field: value })`
- **Example:**
- This query will retrieve all documents from the `articles` collection where the `author` field is equal to "Mary".
`db.Articles.find({ author: "Mary" })`

NOSQL - MONGODB

- **What Does a MongoDB Query Return?**

- A MongoDB query returns a **cursor** object that points to the documents that match the query conditions. The cursor can be iterated over to access each document. You can also use the **.pretty() method** to format the output in a readable way. If no matching documents are found, the query will return an empty result.

- I. **Select a Single Document in a Collection**

- To select a single document in MongoDB, the `findOne()` method is used. It searches for a document that matches the given **filter criteria** and returns the first document that meets the condition. If no document matches the query, it returns null. This is useful when you want to retrieve a **specific document**, like finding a user by their **unique identifier** or looking up a product by its ID.

- **Syntax:**

```
db.collection_name.findOne()
```

NOSQL - MONGODB

2. Select All Documents in a Collection

- To retrieve all documents from a collection in **MongoDB**, the **find()** method is used. This method returns a cursor that iterates through all documents in the collection. By default, **find()** will return all **documents** without any **filters**, but you can add conditions to limit the results. You can also use **.pretty()** to format the output for better readability.

- **Example:**

```
db.collection_name.find().pretty()
```

3. Filter Documents with Equality Condition

- The equality operator (**\$eq**) is used to match the documents where the value of the field is equal to the specified value. In other words, the **\$eq** operator is used to specify the equality condition.

```
db.collection_name.find({ field: { $eq: value } })
```

NOSQL - MONGODB

4. Filter Documents with Greater Than or Less Than Condition

- To **filter documents** based on numeric conditions, such as greater than or less than, you can use the \$gte (greater than or equal) and \$lte (less than or equal) operators within the find() method. These operators allow you to retrieve documents that meet specific criteria based on **numeric values**.

- **Syntax:**

```
db.collection_name.find({ field: {$gte : value }})
or
db.collection_name.find({ field: {$lte : value }})
```

5. Check the Existence of a Field

- The \$exists operator in MongoDB is used to check whether a specific field exists in the **documents** of a **collection**. This operator helps filter documents that either have or do not have a particular field, providing more flexibility in queries.

```
db.collection_name.find({ field: {$exists : boolean }})
```

NOSQL - MONGODB

- **Advanced MongoDB Query Techniques**
- **AND Condition**
- The **\$and operator** is a MongoDB logical operator that performs a logical AND on an array of one or more expressions. It retrieves only those documents that satisfy **all** the specified expressions in the array.

```
db.collection_name.find({  
    $and: [  
        { condition1 },  
        { condition2 },  
        ...  
    ]  
})
```

NOSQL - MONGODB

- **OR Condition**

- The \$or operator in MongoDB is used to retrieve documents that match **any of the specified conditions**. It is helpful when you need to match multiple conditions and retrieve documents that meet **at least one of them**.

```
db.collection_name.find({  
  $or: [  
    { condition1 },  
    { condition2 },  
    ...  
  ]  
})
```

- **Limit the Number of Results**

- The limit() method in MongoDB allows you to specify the **maximum number of documents** that should be returned in a query result. This is useful when you want to restrict the number of documents **processed** or **displayed**.

```
db.collection_name.find(query).limit(number)
```

NOSQL - MONGODB

- **What is MongoDB Compass?**
- MongoDB Compass is a **GUI tool** that provides a visual way to interact with MongoDB databases, making database management intuitive and efficient. It is designed to cater to developers of all skill levels, offering an accessible and user-friendly interface. It allows developers to:
 - Explore and manage database schemas, collections and documents.
 - Perform complex queries without writing code manually.
 - Visualize data patterns with charts and graphs.
 - Monitor real-time database performance and ensure data integrity with validation rules.



NOSQL - MONGODB

- **Features of MongoDB Compass**
- **Visual Representation:** MongoDB Compass visually represents database schemas, collections and documents, making it easier to understand the data structure.
- **Query Builder:** Its query builder allows users to construct complex queries using a point-and-click interface and eliminating the need for manual MongoDB query syntax.
- **Index Management:** MongoDB Compass simplifies index management, enhancing database performance by allowing users to create, view, and analyze indexes.
- **Data Visualization:** Built-in data visualization tools enable users to analyze data using charts, graphs and histograms.
- **Real-Time Performance Monitoring:** MongoDB Compass allows users to monitor database performance in real-time, helping identify and resolve bottlenecks promptly.
- **Document Validation:** Compass includes a document validation feature that enables users to define and enforce data validation rules ensuring data integrity.
- **Data Exploration:** It offers tools for filtering, sorting, and aggregating data.

NOSQL - MONGODB

- **MongoDB CRUD Operations**
- CRUD operations are essential for interacting with databases. In MongoDB, CRUD operations allow users to perform various actions like inserting new documents, reading data, updating records and deleting documents from collections.
 - **Create:** Add new documents to a collection.
 - **Read:** Retrieve documents from a collection.
 - **Update:** Modify existing documents.
 - **Delete:** Remove documents from a collection.

NOSQL - MONGODB

I. Create Operations

- The create or insert operations are used to insert or add new documents in the collection. If a collection does not exist, then it will create a new collection in the database. We can perform create operations using the following methods provided by MongoDB:

Method	Description
<code>db.collection_name.insertOne()</code>	It is used to insert a single document in the collection.
<code>db.collection_name.insertMany()</code>	It is used to insert multiple documents in the collection.
<code>db.createCollection()</code>	It is used to create an empty collection.

NOSQL - MONGODB

```
db.Student.insertMany([
{
    name    : "Jack",
    age     : 20,
    course  : "C++ Programming",
},
{
    name    : "Robin",
    age     : 21,
    course  : "Web Design",
}
])
{
    "acknowledged" : true,
    "insertedIds" : [ ObjectId("5e540d3192e6dfa3fc48ddaf"), ObjectId("5e540d3192e6dfa3fc48ddb0") ]
}
```

NOSQL - MONGODB

2. Read Operations

- The Read operations are used to retrieve documents from the collection, or in other words, read operations are used to query a collection for a document. We can perform read operations using the following method provided by MongoDB:

Method	Description
<code>db.collection_name.find()</code>	It is used to retrieve all documents from the collection.
<code>db.collection_name.findOne()</code>	Retrieves a single document that matches the query criteria.

NOSQL - MONGODB

3. Update Operations

- The update operations are used to update or modify existing documents in the collection. We can update a single document or multiple documents that match a given query. We can perform update operations using the following methods provided by MongoDB:

Method	Description
<code>db.collection_name.updateOne()</code>	It is used to update a single document in the collection that satisfies the given criteria.
<code>db.collection_name.updateMany()</code>	It is used to update multiple documents in the collection that satisfy the given criteria.
<code>db.collection_name.replaceOne()</code>	It is used to replace a single document in the collection that satisfies the given criteria.

NOSQL - MONGODB

- **Example 1:** In this example, we are updating the age of Jack in the Student collection using **db.collection_name.updateOne()** method.

```
db.Student.updateOne({ name: "Jack" }, { $set: { age: 21 } })
```

- **Example 2:** In this example, we are updating the course in all the documents in the Student collection using **db.collection_name.updateMany()** method.

```
db.Student.updateMany({}, { $set: { course: "C++ Programming" } })
```

NOSQL - MONGODB

4. Delete Operations

- The delete operations are used to delete or remove documents from a collection. We can delete documents based on specific criteria or remove all documents. We can perform delete operations using the following methods provided by MongoDB:

Method	Description
<code>db.collection_name.deleteOne()</code>	It is used to delete a single document from the collection that satisfies the given criteria.
<code>db.collection_name.deleteMany()</code>	It is used to delete multiple documents from the collection that satisfy the given criteria.

NOSQL - MONGODB

- **Example 1:** In this example, we are deleting a document from the Student collection using **db.collection_name.deleteOne()** method.
`db.Student.deleteOne({ name: "Robin" })`
- **Example 2:** In this example, we are deleting all the documents from the Student collection using **db.collection_name.deleteMany()** method.
`db.Student.deleteMany({})`